

基于 CIFAR-10 数据集的深度卷积条件对抗生成网络

——DC-CGAN 设计与应用

SC24219070 朱越： 课题设计及模型设计

SC24219073 向旭锦： 模型训练及模型评价

SC24219049 陈蒙： 方案论证和报告制作

写在前面：本文工作参考 Pytorch-GAN 仓库中的 `cgan.py` 和 `dcgan.py` 实现方法，在 `cgan.py` 文件基础上改编，因此沿用了该文件中的程序架构与训练策略。本文实验尽可能使用轻量化模型实现较高性能的图片生成网络。

第一章 DC-CGAN 的设计与训练

1. 课题背景：

(1) Pytorch-GAN 仓库：

Pytorch-GAN 仓库收集了多种生成对抗网络(GAN)的 PyTorch 实现，包括条件对抗生成网络 CGAN、深度卷积对抗生成网络 DCGAN、BEGAN、BicycleGAN、边界寻求 GAN 等多种模型，并包含不同模型的实现代码、模型权重、示例图像等。其中，CGAN 和 DCGAN 为适用于小规模数据的图像对抗生成网络，仓库提供了其在 MNIST 数据集上的训练脚本和训练结果与日志。

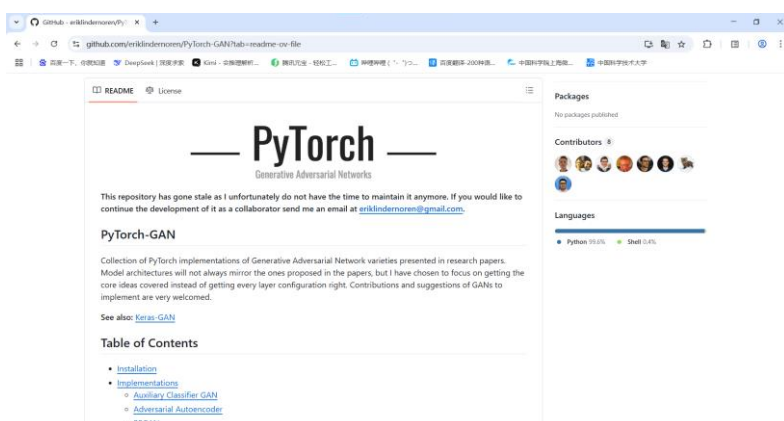


图 1 Pytorch-GAN 仓库首页

(2) 条件对抗生成网络(CGAN)：

普通的 GAN 网络在生成高维度数据时，通常只能随机生成，无法生成我们给定条件的数据，这极大的限制了 GAN 的应用范围。而只有能够生成我们指定条件的数据，这会更有意义。CGAN-条件生成网络是在生成器和判别器二者相互对抗训练的基础上引入了条件信息 c ，这可以是类别标签或者是其他类型的数据，生成器和鉴别器的输入都在原数据基础上加入了条件信息，将类型标签（条件信息）与生成的图像信息进行关联，从而生成满足特定条件的数据样本。

在 Pytorch-GAN 仓库提供的 CGAN 训练代码中，使用完全的全连接层网络 (MLP)来构建生成器与鉴别器，在 MNIST 数据集上取得了有效的结果。

(3) 深度卷积对抗生成网络(DCGAN):

深度卷积对抗生成网络，通过将卷积神经网络引入到生成器和判别器中（但是不包括池化层），并替代了原有 GAN 网络的全连接层；判别器经过一系列卷积进行特征提取，通过卷积下采样，最终输出一个概率值表示输入图像是真实图像的概率；生成器将卷积层替换成了反卷积层，来逐渐增加图像的尺寸和深度；为了防止过拟合，判别器和生成器中使用批归一化层，降低参数初始化的影响，有助于加速模型训练，提升训练的稳定性，并使用恰当的激活函数，提高了图片场景里训练的稳定性，生成的图片质量较高。

然而，DCGAN 不具备条件生成功能，一方面无法控制生成图像的内容，另一方面可能会存在对于数据集的学习偏向某一类标签的问题。

(4) CIFAR-10 数据集:

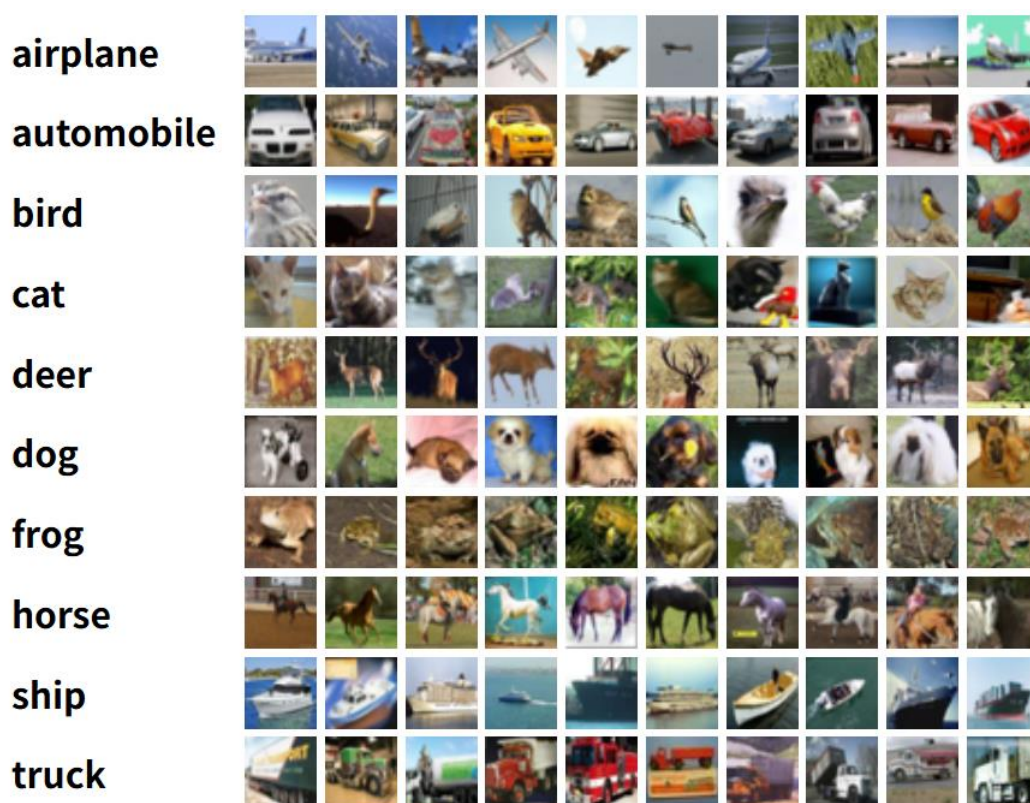


图 2 CIFAR-10 数据集

CIFAR-10(Canadian Institute For Advanced Research - 10)数据集是一个常用于机器学习和计算机视觉领域的经典数据集，每张图像大小为 32×32 像素，具有 3 个颜色通道，属于低分辨率图像。数据集包含 10 个互不重叠的类别：飞机(airplane)，汽车(automobile)，鸟类(bird)，猫(cat)，鹿(deer)，狗(dog)，青蛙(frog)，马(horse)，船(ship)，卡车(truck)，每个类别有 6000 张图像，在训练集和测试集中均匀分布，即每类在训练集中有 5000 张图像，在测试集中有 1000 张图像。

2. 技术路线:

(1) 生成器设计:

Pytorch-GAN 仓库中提供的 CGAN 实现虽然在 MNIST 数据集上取得了一定的结果,但在 CIFAR-10 数据集上的表现却较为一般。这是由于 MLP 网络不擅长图像信息的提取,而 CIFAR-10 数据集虽然像素较低,但其特征的形貌、纹理都较为复杂,更适合使用卷积层来处理。

综合 CGAN 和 DCGAN 的特点,为了实现轻量级的 CIFAR-10 条件生成,我们提出将 DCGAN 中的深度卷积块引入 CGAN 当中,实现基本的 DC-CGAN 生成器。在这个过程中,我们主要引入了三个重要设计:

a) 条件批量归一化层:

CGAN 的特征融合策略是直接拼接映射后的标签向量与高维噪声:

```
39 class Generator(nn.Module):
40     def __init__(self):
41         super(Generator, self).__init__()
42
43         self.label_emb = nn.Embedding(opt.n_classes, opt.n_classes)
44
45         def block(in_feat, out_feat, normalize=True):
46             layers = [nn.Linear(in_feat, out_feat)]
47             if normalize:
48                 layers.append(nn.BatchNorm1d(out_feat, 0.8))
49             layers.append(nn.LeakyReLU(0.2, inplace=True))
50             return layers
51
52         self.model = nn.Sequential(
53             *block(opt.latent_dim + opt.n_classes, out_feat=128, normalize=False),
54             *block(in_feat=128, out_feat=256),
55             *block(in_feat=256, out_feat=512),
56             *block(in_feat=512, out_feat=1024),
57             nn.Linear(1024, int(np.prod(img_shape))),
58             nn.Tanh()
59         )
60
61     def forward(self, noise, labels):
62         # Concatenate label embedding and image to produce input
63         gen_input = torch.cat((self.label_emb(labels), noise), -1)
64         img = self.model(gen_input)
65         img = img.view(img.size(0), *img_shape)
66         return img
```

我们在第一次实验中直接沿用了这种方式,制作了

$$dim = latent_dim + classes_dim$$

的特征图。然而这种方式在训练中迅速产生了模式崩溃的结果,这是由于相对于噪声维度来说,条件维度占比太小,导致生成器陷入局部特解中。

因此我们在第二次实验中提高了条件维度,使

$$target_dim = latent_dim$$

这种特征图在 10 轮训练之后就可以展现出不同的类间差异,但是类内差异有限,这是由于噪声与条件没有深度融合,导致条件成为生成种子的主要部分:



图 3 严重的类内多样性不足

我们经过多次探索，引入了条件批量归一化层深度融合条件信息与噪声种子：

```

38 # 条件批归一化层（核心修改）
39 class ConditionalBatchNorm2d(nn.Module): 2用法
40     def __init__(self, num_features, cond_size):
41         super().__init__()
42         self.bn = nn.BatchNorm2d(num_features, affine=False)
43         self.gamma = nn.Linear(cond_size, num_features)
44         self.beta = nn.Linear(cond_size, num_features)
45
46     def forward(self, x, cond):
47         normalized = self.bn(x)
48         gamma = self.gamma(cond).view(x.size(0), -1, 1, 1)
49         beta = self.beta(cond).view(x.size(0), -1, 1, 1)
50         return gamma * normalized + beta

```

条件批量归一化层通过映射条件特征计算归一化参数，实现基于特征的特征图归一化操作，可以有效地将条件与噪声融合，同时增强类间差异和类内多样性。

b) 加权特征融合层：

在使用条件批归一化后，生成器产生的图像确实产生了部分差异，然而仍不能称之为“多样性”。因此，我们又引入了加权特征融合层：

```

57 self.label_emb = nn.Embedding(opt.n_classes, opt.latent_dim)
58
59 self.init_size = opt.img_size // 4
60 # 条件投影层
61 self.l1 = nn.Sequential(
62     nn.Linear(opt.latent_dim, 512),
63     nn.BatchNorm1d(512),
64     nn.LeakyReLU(0.2)
65 )
66
67 # 初始特征图生成
68 self.l2 = nn.Linear(512, 128 * self.init_size ** 2)

```



```

88     def forward(self, noise, labels):
89         # Concatenate label embedding and image to produce input
90         gen_labels = self.l1(self.label_emb(labels)) # [batch, 512]
91         gen_noise = self.l1(noise) # [batch, 512]
92         out = 0.6 * gen_labels + 0.4 * gen_noise
93
94         # 初始特征图
95         img = self.l2(out)
96         img = img.view(img.shape[0], 128, self.init_size, self.init_size) # [batch, 128, 4, 4]

```

加权特征融合层通过分离的条件与噪声特征映射，得到两个相互独立的特征向量，然后使用归一化的加权系数融合两个特征向量，控制加权系数的权重，可以改变条件注入程度，找到条件与噪声的“黄金配比”，实现类内多样性提高。



图 4 具有加权特征融合的生成器

c) 层间特征/噪声增强层:

在使用加权特征融合后，生成器性能得到了较大的提升。但是在多轮训练后，生成器在对抗中逐渐趋于劣势，仍会试图使用局部特解，这显然不利于网络训练的长期稳定和优化。因此，我们又引入了层间增强：

```

98     # 上采样 + 条件注入 + 噪声注入
99     img = self.block1(img) # [batch, 128, 8, 8]
100     img = self.cbn1(img, out)
101     img = img + 0.01 * torch.randn_like(img) # 添加层间噪声
102
103     img = self.block2(img) # [batch, 64, 16, 16]
104     img = self.cbn2(img, out)
105     img = img + 0.05 * torch.randn_like(img) # 添加层间噪声
106
107     img = self.final(img) # [batch, 3, 32, 32]

```

层间增强块旨在强制改善网络生成图像的条件和噪声信息，迫使生成器在每一层都优化图像，而不是在更深的层中趋于同质化。从预训练结果可以看出，三重优化的生成器已经可以生成具有较强分辨力的图像了。

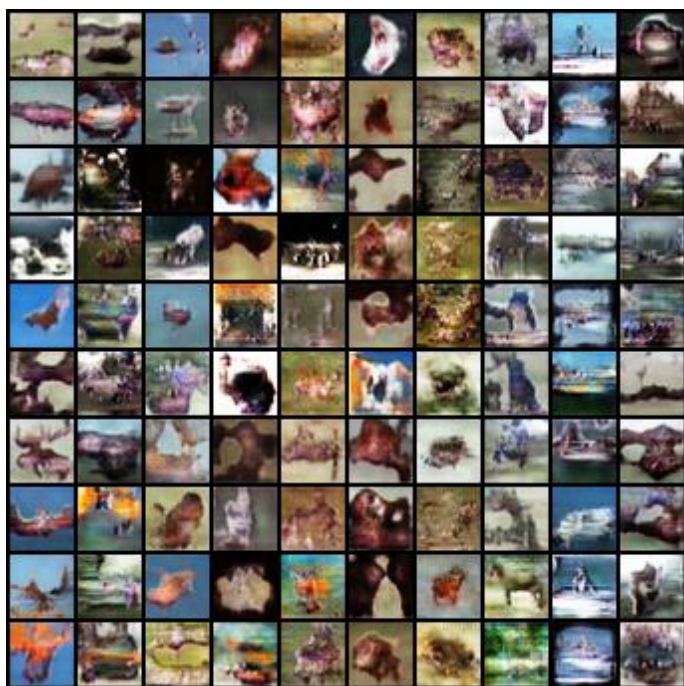


图 5 具有较优性能的生成器结果

融合了三项重要优化的生成器如图所示：

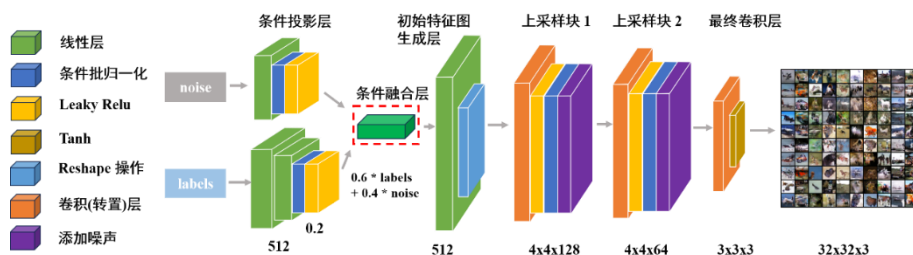


图 6 DC-CGAN 生成器

```

38 # 条件批归一化层（核心修改）
39 class ConditionalBatchNorm2d(nn.Module): 2用法
40     def __init__(self, num_features, cond_size):
41         super().__init__()
42         self.bn = nn.BatchNorm2d(num_features, affine=False)
43         self.gamma = nn.Linear(cond_size, num_features)
44         self.beta = nn.Linear(cond_size, num_features)
45
46     def forward(self, x, cond):
47         normalized = self.bn(x)
48         gamma = self.gamma(cond).view(x.size(0), -1, 1, 1)
49         beta = self.beta(cond).view(x.size(0), -1, 1, 1)
50         return gamma * normalized + beta
51
52
53 class Generator(nn.Module):
54     def __init__(self):
55         super(Generator, self).__init__()
56
57         self.label_emb = nn.Embedding(opt.n_classes, opt.latent_dim)
58
59         self.init_size = opt.img_size // 4
60         # 条件投影层
61         self.l1 = nn.Sequential(
62             nn.Linear(opt.latent_dim, 512),
63             nn.BatchNorm1d(512),
64             nn.LeakyReLU(0.2)
65         )
66
67         # 初始特征图生成
68         self.l2 = nn.Linear(512, 128 * self.init_size ** 2)

```

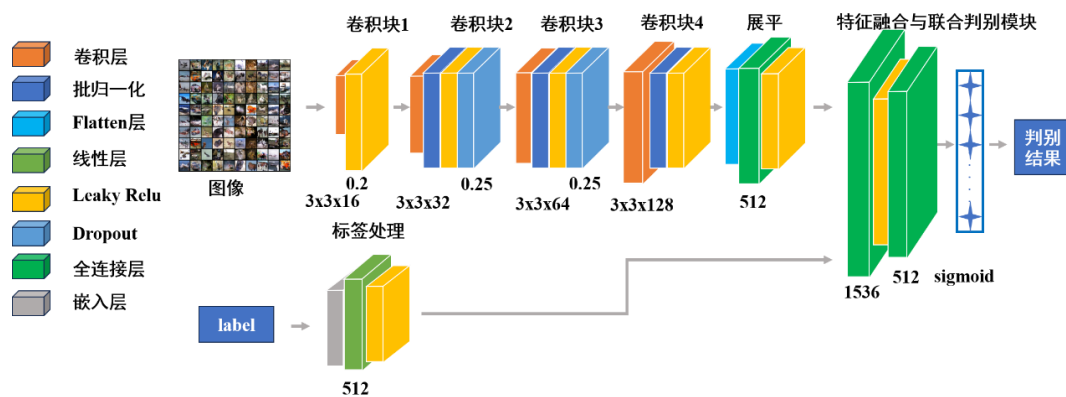
```

70 # 上采样块 (使用条件批归一化)
71 self.block1 = nn.Sequential(
72     nn.ConvTranspose2d(128, 128, 4, stride=2, padding=1, bias=False),
73     nn.LeakyReLU(0.2)
74 )
75 self.cbn1 = ConditionalBatchNorm2d( num_features= 128, cond_size= 512)
76
77 self.block2 = nn.Sequential(
78     nn.ConvTranspose2d(128, 64, 4, stride=2, padding=1, bias=False),
79     nn.LeakyReLU(0.2)
80 )
81 self.cbn2 = ConditionalBatchNorm2d( num_features= 64, cond_size= 512)
82
83 self.final = nn.Sequential(
84     nn.Conv2d(64, opt.channels, 3, stride=1, padding=1),
85     nn.Tanh()
86 )
87
88 def forward(self, noise, labels):
89     # Concatenate label embedding and image to produce input
90     gen_labels = self.l1(self.label_emb(labels)) # [batch, 512]
91     gen_noise = self.l1(noise) # [batch, 512]
92     out = 0.6 * gen_labels + 0.4 * gen_noise
93
94     # 初始特征图
95     img = self.l2(out)
96     img = img.view(img.shape[0], 128, self.init_size, self.init_size) # [batch, 128, 4, 4]
97
98     # 上采样 + 条件注入 + 噪声注入
99     img = self.block1(img) # [batch, 128, 8, 8]
100     img = self.cbn1(img, out)
101     img = img + 0.01 * torch.randn_like(img) # 添加层间噪声
102
103     img = self.block2(img) # [batch, 64, 16, 16]
104     img = self.cbn2(img, out)
105     img = img + 0.05 * torch.randn_like(img) # 添加层间噪声
106
107     img = self.final(img) # [batch, 3, 32, 32]
108     return img

```

(2) 鉴别器设计:

鉴别器的设计秉承与生成器对称的原则，分别处理输入条件与图像，同时提取高维特征向量后注入联合判别器，最终输出二元结果。其结构图如下：



```

78 class Discriminator(nn.Module):
79     def __init__(self):
80         super(Discriminator, self).__init__()
81
82     def discriminator_block(in_filters, out_filters, bn=True, dp=True):
83         block = [nn.Conv2d(in_filters, out_filters, 3, 2, 1)]
84         if bn:
85             block.append(nn.BatchNorm2d(out_filters, 0.8))
86         block.append(nn.LeakyReLU(0.2, inplace=True))
87         if dp:
88             block.append(nn.Dropout2d(0.25))
89         return block

```

```

91     # 图像特征提取
92     self.img_encoder = nn.Sequential(
93         # 输入: (3, 32, 32)
94         *discriminator_block(opt.channels, out_filters: 16, bn=False), # 输出: (16, 16, 16)
95         *discriminator_block(in_filters: 16, out_filters: 32), # 输出: (32, 8, 8)
96         *discriminator_block(in_filters: 32, out_filters: 64), # 输出: (64, 4, 4)
97         *discriminator_block(in_filters: 64, out_filters: 128, dp=False), # 输出: (128, 2, 2)
98
99         nn.Flatten(), # 输出: (batch, 128*2*2) = (batch, 512)
100         nn.Linear(512, 512),
101         nn.LeakyReLU(0.2)
102     )
103
104     # 标签处理路径
105     self.label_embedding = nn.Sequential(
106         nn.Embedding(opt.n_classes, 512),
107         nn.Linear(512, 512),
108         nn.LeakyReLU(0.2, inplace=True)
109     )
110
111     # 联合鉴别器 (维度对齐)
112     self.model = nn.Sequential(
113         nn.Linear(1536, 512), # 512(img) + 512(label) + 512(cross) = 1536
114         nn.LeakyReLU(0.2, inplace=True),
115         nn.Linear(512, 1),
116         nn.Sigmoid()
117     )
118
119     def forward(self, img, labels):
120         # 处理图像
121         img_in = self.img_encoder(img) # [batch, 512]
122         # 处理类别
123         label_in = self.label_embedding(labels) # [batch, 512]
124
125         # 特征融合 (维度对齐)
126         d_in = torch.cat([img_in, label_in, img_in * label_in], dim=1) # [batch, 1536]
127         validity = self.model(d_in)
128         return validity

```

(3) 三批次渐进式训练策略:

在生成器设计方面，我们进行了针对图像特征和多样性的优化，在训练中，我们也使用了一种多批次渐进的训练策略，旨在得到更加稳定收敛的训练结果：

a) 第一批训练（预训练）:

在初步训练中，我们首先改进了原始脚本中的单一学习率，采用分离学习率 ($lr_g = 2e-4$, $lr_d = 1e-4$) 来控制生成器与鉴别器的优化速率。为了避免模式崩溃，我们希望在早期训练中条件特征占据优势，因此我们将条件特征加权系数设置为 0.6（噪声特征加权系数为 0.4），略高于均值。

同时，根据鉴别器先行原则，我们在这批次的训练中引入了竞争模式，目的是鼓励鉴别器争取更多的训练机会，使其具有充分的鉴别能力。具体设计为：

```

281     # -----
282     # Train Discriminator
283     # -----
284
285     for _ in range(d_steps):
286         optimizer_D.zero_grad()
287
288         # Loss for real images
289         validity_real = discriminator(real_imgs, labels)
290         d_real_loss = adversarial_loss(validity_real, valid - 0.05)
291
292         # Loss for fake images
293         validity_fake = discriminator(gen_imgs.detach(), gen_labels)
294         d_fake_loss = adversarial_loss(validity_fake, fake + 0.05)
295
296         # Total discriminator loss
297         d_loss = (d_real_loss + d_fake_loss) / 2
298
299         d_loss.backward()
300         optimizer_D.step()

```


而对于生成器，我们使用退避策略，强制其等待鉴别器优化，避免竞争机制失效：

```
256 # Configure input
257 real_imgs = Variable(imgs.type(FloatTensor))
258 labels = Variable(labels.type(LongTensor))
259
260 # -----
261 # Train Generator
262 # -----
263
264 optimizer_G.zero_grad()
265
266 # Sample noise and labels as generator input
267 z = Variable(FloatTensor(np.random.normal(0, 1, (imgs.shape[0], opt.latent_dim))))
268 gen_labels = Variable(LongTensor(np.random.randint(0, opt.n_classes, imgs.shape[0])))
269
270 # Generate a batch of images
271 gen_imgs = generator(z, gen_labels)
272
273 # Loss measures generator's ability to fool the discriminator
274 validity = discriminator(gen_imgs, gen_labels)
275 g_loss = adversarial_loss(validity, valid)
276
277 if i % g_steps == 0:
278     g_loss.backward()
279     optimizer_G.step()
```

具体的竞争退避策略如下：

```
302 g_steps = int(torch.ceil(torch.abs(g_loss - 1.0) * 10).clamp(1, 3))
303 d_steps = int(torch.ceil(torch.abs(d_loss - 0.5) * 10).clamp(1, 3))
```

在这种模式下，鉴别器会在训练初期迅速掌握图像特征和纹理，并使 d_loss 迅速接近目标值；接近目标值后，鉴别器竞争欲望下降，并接受生成器退避，此时生成器开始接受鉴别器的优化建议，使 g_loss 开始接近目标值；当生成器具有一定的生成能力后，两者开始公平对抗，几乎不再参与竞争退避策略，第一批训练结束：

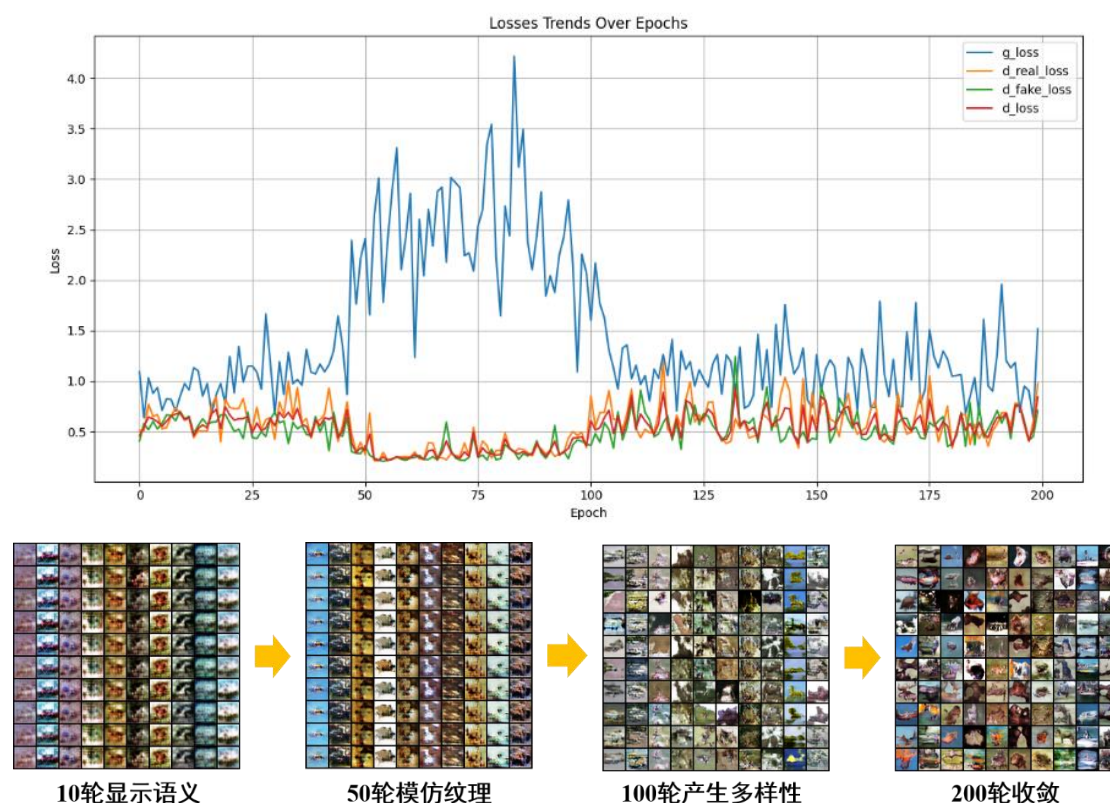


图 8 第一批次训练情况

b) 第二批训练（初步优化）:

在第一批训练后期，我们可以看到 g_loss 振荡较为明显，继续当前策略训练可能收效甚微，因此我们在第二批训练中采用一定的优化策略。

首先我们摒弃了竞争退避的方式，平衡特征融合权重（均设置为 0.5），然后使用了更小的学习率，希望训练逐步进入收敛：

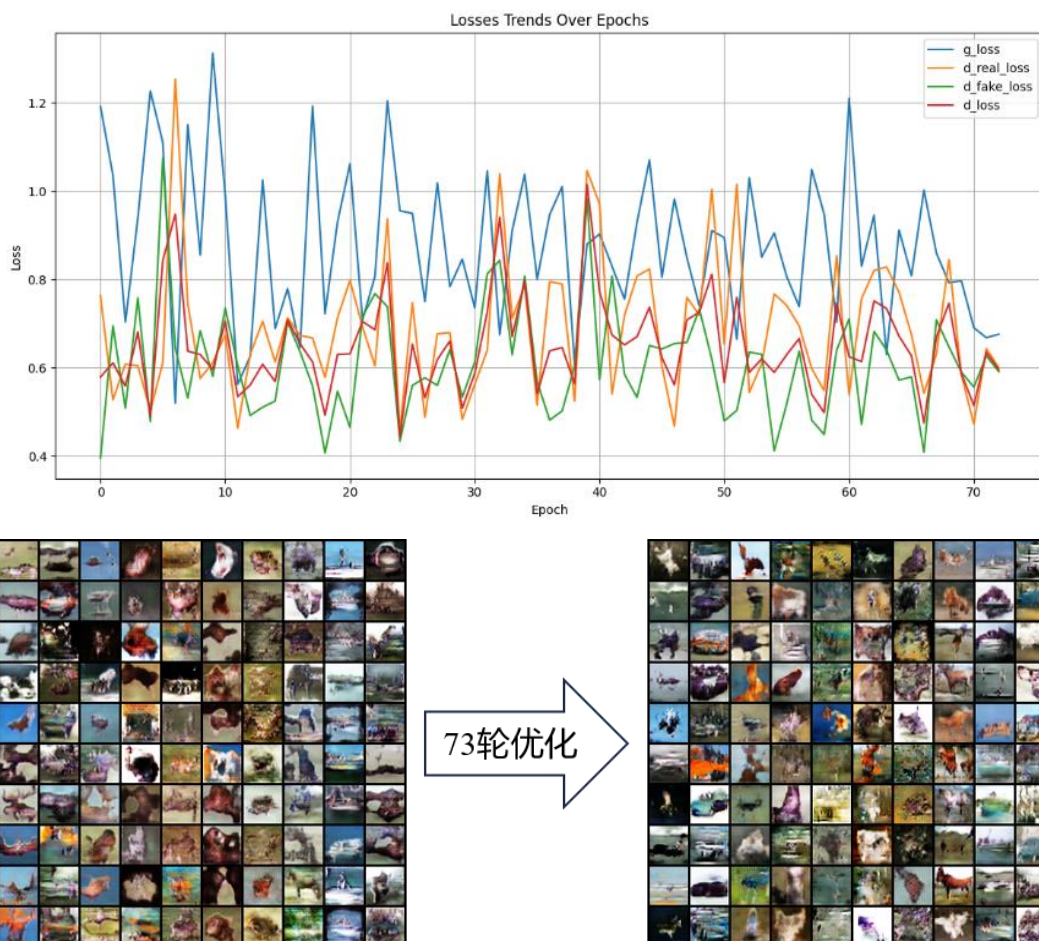


图 9 第二批训练情况

在这一批次的训练中，我们开始使用早停条件，目的是在训练行为符合预期后迅速进入下一阶段的优化。结果表明，这一批次在 73 轮就达成目标了。

```
313     if (epoch >= 10
314         and torch.mean(torch.std(loss_history[epoch-10:epoch, :], dim=1)) < 0.12
315         and torch.std(torch.mean(loss_history[epoch-10:epoch, :], dim=1)) < 0.15
316     ):
317         sample_image(n_row=10, batches_done=epoch + 1)
318         break
```

c) 第三批训练（进一步优化）:

经过第二批训练后，网络对抗进一步收敛，但仍不是最优状态。因此，我们又引入了第三批训练。这一批次采用了更小的学习率，并略微增强了噪声特征的加权系数，使得最终生成器的多样性占优：

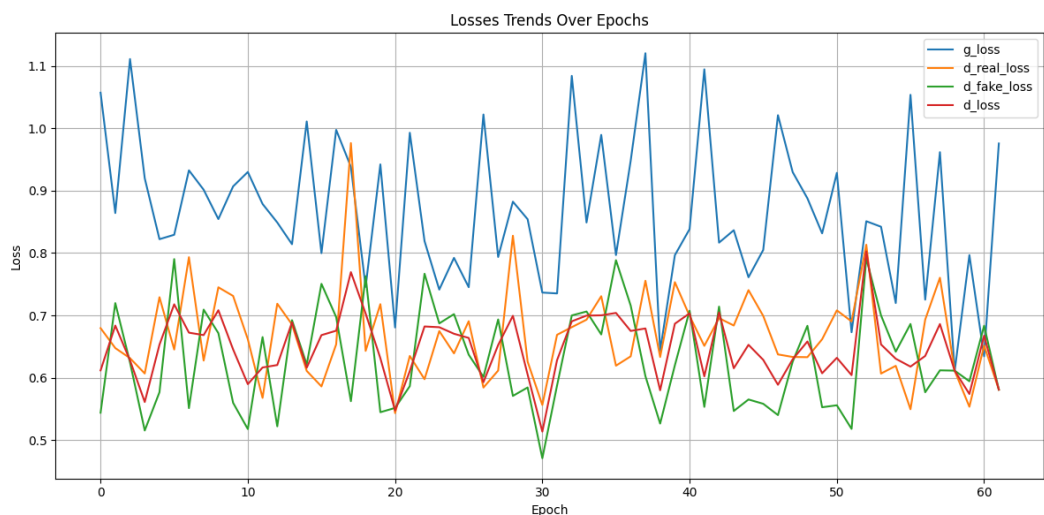


图 10 第三批训练情况

同时，我们设计的早停条件为纳什均衡点(loss = 0.693)，结果表明，本批次的训练在 62 轮就达到目标了。

(4) IS 和 FID 评估：

在评估环节，我们使用的是 pytorch 官方的 FrechetInceptionDistance 和 InceptionScore，使用 2048 特征维度和 10000 样本容量的测试结果为：

（Deepseek 害人不浅，它给的参数完全不能用）

表 1 DC-CGAN 评价指标及对比

网络类型	生成器参数量	FID 指标*	IS 指标*
CGAN	3,856,612(~3.86M)	247.80	2.41±0.06
DCGAN	1,051,139(~1.05M)	72.44	4.89±0.16
DC-CGAN(non-opt**)	1,304,359(~1.3M)	70.46	5.80±0.14
DC-CGAN	1,696,299(~1.7M)	63.56	5.02±0.08

注：*两项指标均使用相同的脚本计算。

**non-opt 是指未使用三项生成器优化及多批次训练优化的网络结果。

3. 结果与讨论:

综上，我们可以看出，DC-CGAN 确实可以通过引入卷积层大幅减小 CGAN 的全连接层参数；同时，生成器优化以及训练策略优化也可以有效提升 DC-CGAN 生成器的最终表现。

然而，目前的 DC-CGAN 还有很多优化空间：

- (1) 卷积层数与输出表现的权衡：目前的 DC-CGAN 仅使用两层卷积层，事实上普遍应用三层卷积层，但更多卷积层可能反而导致表现下降。
- (2) 优化参数的学习：在 DC-CGAN 生成器设计中，加权特征融合层参数是重要超参数之一，人工调节优化较为困难，可以将其设置为可学习参数，由优化器配置权重。
- (3) 训练策略的优化：在当前的训练策略中，loss 计算较为单一，可以引入更多辅助项来帮助优化器梯度下降及收敛。

第二章 基于 Carlini & Wagner 方法的图像前景提取

1. 课题背景:

图像破损是我们生活中常见的现象,无论是照片、壁画、文物,经过时间的侵蚀都会出现缺失、褪色的情况。

图像修复一直以来都是这些领域研究的热门课题,其核心要求在于通过图像残留部分的信息推断缺失部分的信息,具有较高的连贯性要求——也就是在不影响原有信息的条件下,尽可以补充缺失的部分:



图 11 图像修复案例

图像修复也是神经网络的重要应用之一,目前已经发表了许多基于边缘检测、上下文推理、注意力机制的图像修复网络。在本课题中,我们也希望使用 DC-CGAN 实现轻量化的图像修复工作。而要基于 CIFAR-10 完成图像修复,必须先制作破损的 CIFAR-10 数据集。

2. 前景提取方案设计:

为了完成图像修复,必须保留必要的特征信息,CIFAR-10 数据集恰好提供了 10 分类的图像特征,因此我们设计了一种提取图像前景的图像破碎方式:

(1) 松弛函数设计:

破碎图像本质上是一种攻击行为。在这里,我们选择一种特征可控的条件攻击方式——Carlini & Wagner 方法,CW 方法使用松弛函数控制图像的特征。然而,与一般的攻击模式不同,我们的破碎目标是提取图像的特征前景,因此,我们针对松弛函数做出改变:

$$f(x') = \sum_{i \neq t} |Z(x')_i| - Z(x')_t \quad t = label$$

在这样的松弛策略中,优化器将尽可能增大原类别的识别概率,而迫使其余的类别识别概率趋近于零。这样在优化的过程中,优化器就会保留前景特征,而抛弃与图片特征无关的背景信息。

(2) 掩膜策略及其抛弃率:

在确定松弛函数后,设计抛弃图片像素的掩膜策略。我们使用了“末位淘汰”的掩膜方式,将 CW 方法得到的扰动图像由大到小排序,再将较小部分置 0,然后将不为 0 的部分置 1 形成掩膜,并探索了多种掩膜及其抛弃率的优化策略:

a) 由小到大的抛弃率(drop_rate, dr):

从主观逻辑思维来看，针对图片抛弃像素应当由较小的抛弃率开始，先抛弃不重要的像素，然后逐步收缩，最终保留前景。因此我们的实验从 10%抛弃率开始，随着松弛过程逐步递增。然而，结果并不如我们预期的那样：

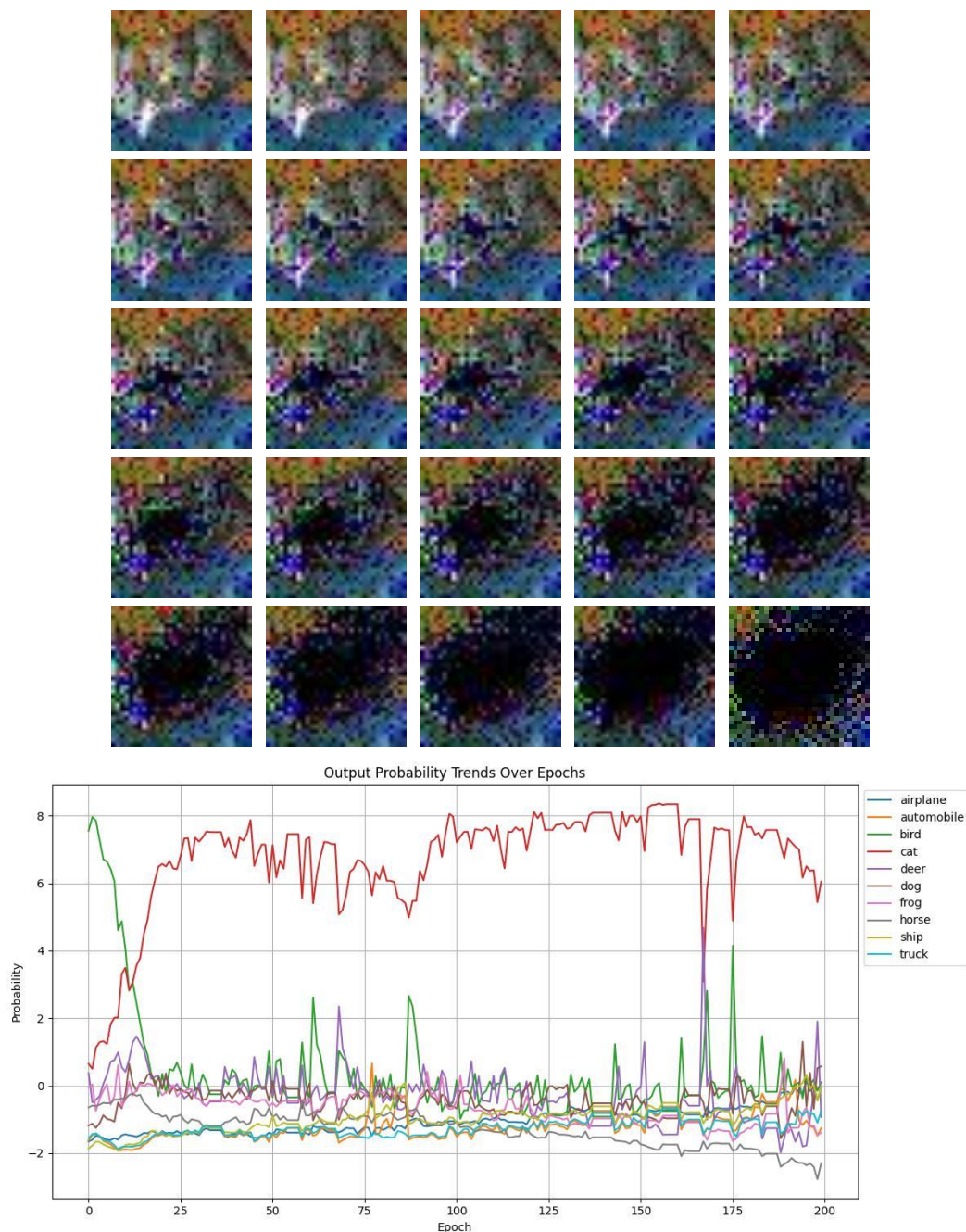


图 12 由小到大的抛弃率

从图中可以看出，由小到大的抛弃率策略反而优先抛弃了图像中的前景信息，保留了大部分的边缘背景。然而根据识别概率变化可以看出，保留了边缘背景的图像却可以实现较高的目标识别率，这样的结果是我们无法理解的。

b) 由大到小的抛弃率:

不过既然我们使用正向的策略得到了相反的结果,我们就可以尝试用反向的策略得到正向的结果。因此,我们尝试使用由大到小的抛弃率,从 90%开始优化:

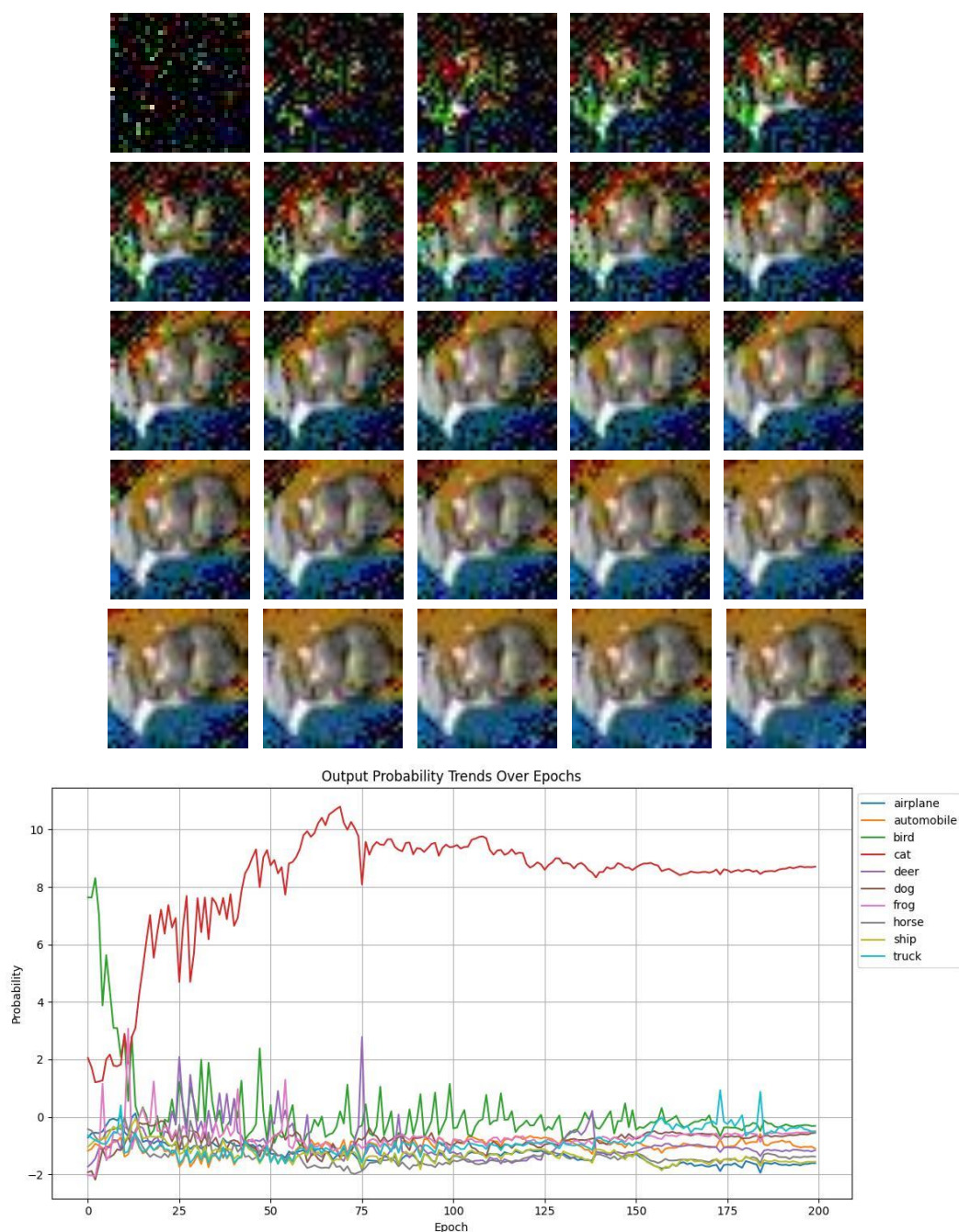


图 13 由大到小的抛弃率

由图可以看出,这次尝试得到了预期的结果。图像掩膜在优化中逐渐尝试标记特征更明显的像素区域,这样将有利于在很少的像素中突出图像的特征,从而使目标的识别率上涨。这种策略显露出图像的前景特征,得到了符合预期的具有可识别特征的残缺图像。

(3) 初步尝试和分析：

实现了基本的前景提取后，我们进一步改善了像素级抛弃策略。我们摒弃了通道分离的掩膜方式，而是综合三通道抛弃情况，判断该图像像素是否直接弃用。具体策略即计算三通道掩膜的均值，将阈值设置为 0.5，这样三通道或二通道保留的均值为 1 与 0.67，则该像素被保留；单通道或无通道保留的均值为 0.33 与 0，则该像素被抛弃。基于这样的策略，我们得到了更接近现实情况的破损图像：

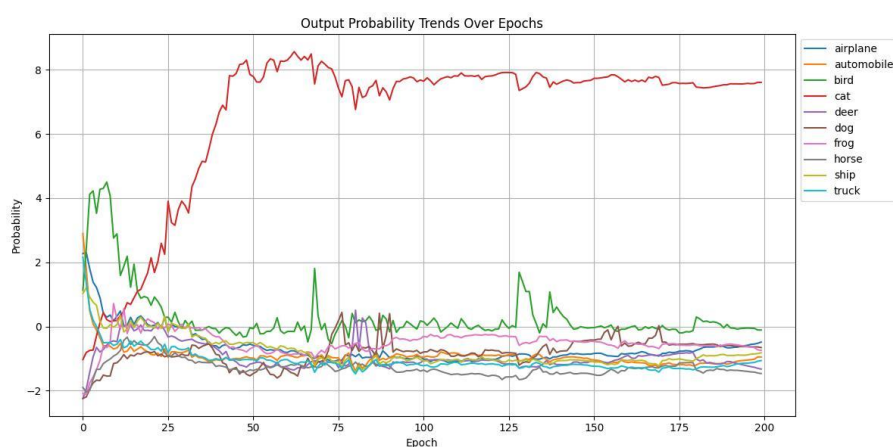


图 14 像素级破损图像

从识别率变化曲线可以看出，事实上有效掩膜优化只在 75 轮之前（上图第二行第 4、5 列）。此时图像能够保留有效前景信息，同时最大程度抛弃前景轮廓外的背景。因此我们设计早停条件，使目标类别识别率占优时抛弃进程停止。

但是，这样做会产生一些较为激进的结果：

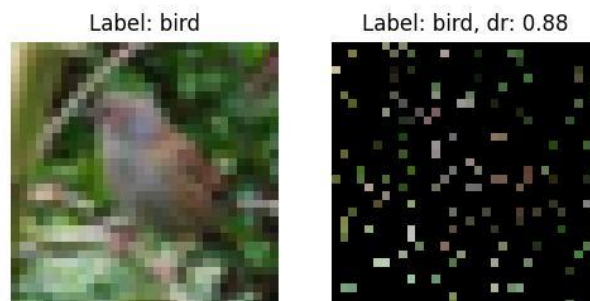


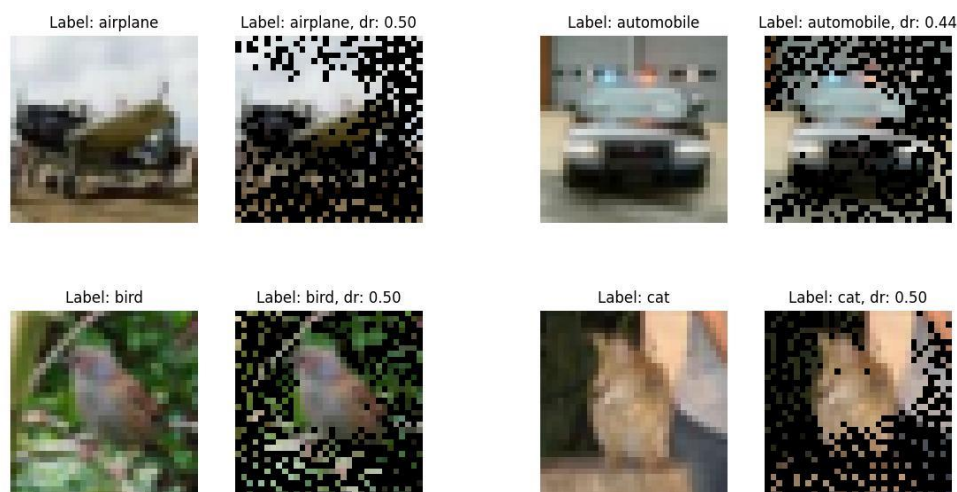
图 15 一重早停条件下激进抛弃率的结果

如图所示，这样的图像虽然利用 CNN 网络识别其类别，但是对人眼来说没有较多可利用的信息，这样的结果没有应用价值，因此不能予以采用。在多次对比后，我们发现 dr 至少要小于 60% 才能保存较为完整的前景信息，因此我们使用二重早停条件：①目标特征的识别率占优；②抛弃率小于 60%。综合二重早停条件，我们可以得到质量较好的破损图像。

3. adv_cifar10 数据集制作：

完成了初步的破碎实验后，就可以开始批量制作破损图像数据集了。为了后续的生成器考虑，我们在这里使用 CIFAR-10 数据集的测试集制作新的小批量样本 adv_cifar10 数据集，并使用 9:1 的训练/测试集比例。

为了实验调用和复用的方便，我们使用 CIFAR-10 数据集的相同格式来保存 adv_cifar10 数据集，以下是部分样例数据：



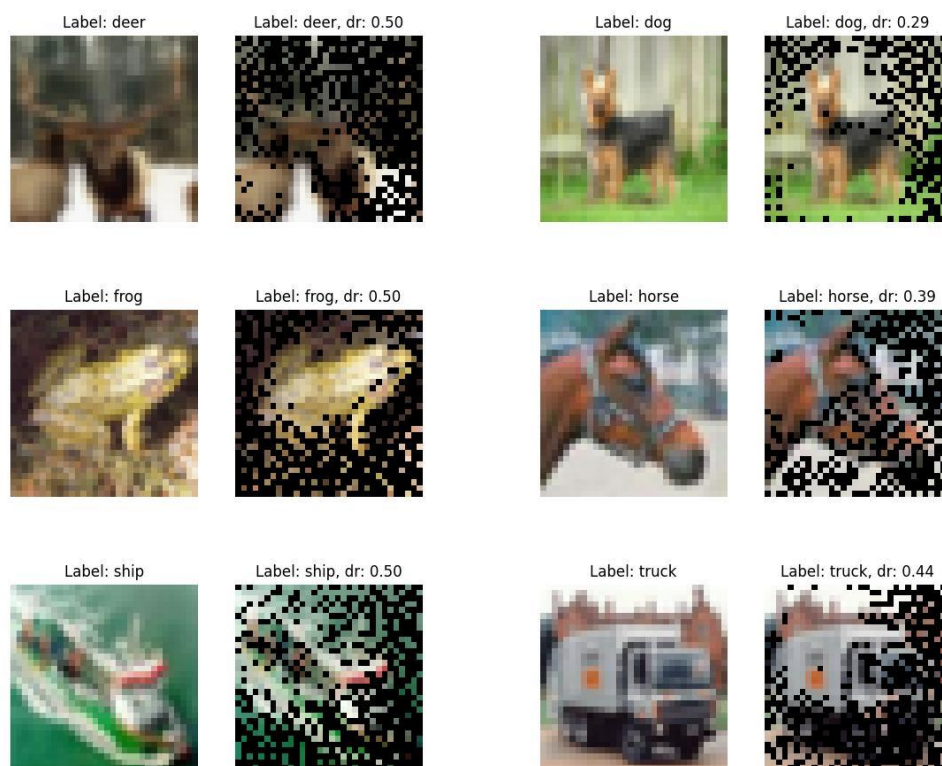


图 16 adv_cifar10 数据集样例

如上图所示，事实证明，基于 CW 的掩膜抛弃像素方法可以有效提取图像的前景特征。

针对 CIFAR-10 测试集的前景提取共计成功制作 9532 张图片，其中划分训练集 8578 张，测试集 954 张。该数据集可以用于训练像素级图像修复生成器。

第三章 基于残差块的掩膜修复生成器(Res-CGAN)设计与训练

1. 从 DC-CGAN 到 Res-CGAN 的优化:

完成 adv_cifar10 数据集的制作后,我们就开始设计针对其的修复生成器。显然,我们目前完成的 DC-CGAN 生成器无法满足修复图像的需求:其在针对图像上采样的过程中会破坏原有图像的信息,并且输入的噪声种子与我们给出图像的格式也不匹配。因此,我们在此基础上进一步改良,设计了一种支持全等变换的掩膜修复生成器。

(1) 引入残差块的生成器:

为了保持图像原有的信息不被破坏,我们参考了残差网络(Resnet)的思想,使用残差通道将图像完好地在层间输送:

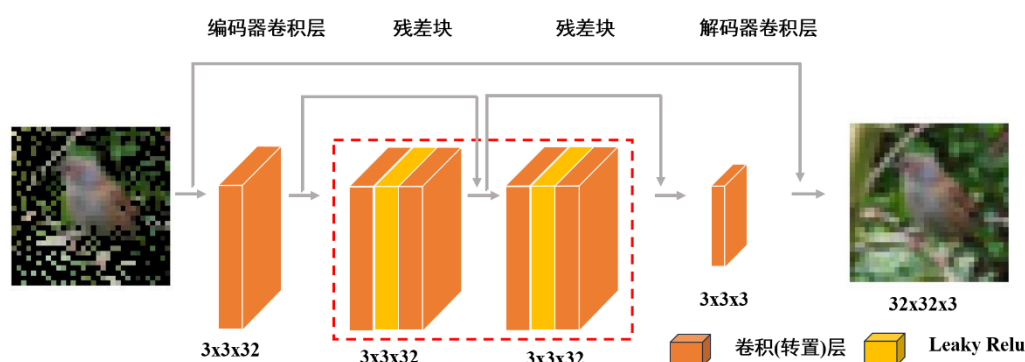


图 17 Res-CGAN 网络结构示意图

在 Res-CGAN 生成器中,我们使用一个残差通道将原始图片完全输送到输出层,并使用掩膜叠加的方式,将改动的像素限制在像素 0 值点,保证原始图像不会受到影响。

在生成路径中,首先使用编码器提取图像特征,然后在生成过程中使用残差连接,使每一层都得到原始图像特征强化,避免后续发生特征丢失等问题。

最后,使用解码器得到背景图像,使用掩膜合成到原始图像当中:

```
38 # 生成器结构
39 class ResBlock(nn.Module):
40     def __init__(self, channels):
41         super().__init__()
42         self.conv1 = nn.Conv2d(channels, channels, 3, padding=1)
43         self.conv2 = nn.Conv2d(channels, channels, 3, padding=1)
44         self.relu = nn.ReLU()
45
46     def forward(self, x, mask):
47         residual = self.relu(self.conv1(x))
48         residual = self.conv2(residual)
49         return x + mask * residual # 仅缺失区域更新
50
51 class Generator(nn.Module):
52     def __init__(self, n_blocks=2):
53         super().__init__()
54         self.encoder = nn.Conv2d(opt.channels, 32, 3, padding=1)
55         self.res_blocks = nn.ModuleList([ResBlock(32) for _ in range(n_blocks)])
56         self.decoder = nn.Conv2d(32, opt.channels, 3, padding=1)
57
58     def forward(self, img, mask):
59         x = self.encoder(img)
60         for block in self.res_blocks:
61             x = block(x, mask)
62         x = self.decoder(x)
63         return (1.0 - mask.float()) * img + mask * x # 合并输出
```

(2) 联合判别训练策略：

针对这样的新生成器，我们在对抗中又采取了一种新的联合判别器策略：

```
263     # -----
264     # Train Generator
265     # -----
266
267     optimizer_G.zero_grad()
268
269     # Generate a batch of images
270     mask = (z == 0)
271     gen_imgs = generator(z, mask)
272
273     # Loss measures generator's ability to fool the discriminator
274     validity = discriminator(gen_imgs)
275     d_loss = adversarial_loss(validity, valid)
276
277     c_loss = 0
278     for i, (gen_img, label) in enumerate(zip(gen_imgs.expand(-1, 3, -1, -1), gen_labels)):
279         # 原始损失计算
280         img = gen_img * std_tensor + mean_tensor
281         logits = net(img)
282         logits_target = logits[:, label]
283         logits_other = logits.abs()[:, [i for i in range(10) if i != label]].sum(1)
284         c_loss = c_loss + logits_other - logits_target
285     c_loss = c_loss / gen_imgs.shape[0]
286
287     g_loss = c_loss * 0.5 + d_loss * 0.5
288     g_loss.backward()
289     optimizer_G.step()
```

在联合鉴别器中，我们使用一个无条件鉴别器和一个分类器同时评价生成图像。其中，无条件鉴别器评价图像纹理是否符合真实图片分布，分类器评价图像是否改变了原有的特征，最终，将两项 loss 值加权得到最终的 loss。在这个过程中，两个判别器本身都不执行优化。

同时，在 adv_cifar10 数据集的导入中使用灰度化（三通道→单通道），试图进一步缩小网络参数量和训练时间：

```
157     dataloader = torch.utils.data.DataLoader(
158         AdvCIFAR10(
159             root='./datas/adv_cifar10',
160             train=True,
161             transform=transforms.Compose([
162                 transforms.Resize(opt.img_size),
163                 transforms.ToTensor(),
164                 transforms.Grayscale(num_output_channels=1),
165                 # transforms.Normalize([0.5], [0.5])
166             ])
167         ),
168         batch_size=opt.batch_size,
169         shuffle=True,
170     )
```

我们使用 resnet56_cifar10 预训练网络权重作为分类器，同时也是 adv_cifar10 数据集制作时 CW 方法的白盒网络。其在原始 CIFAR-10 数据集的识别准确率为 95.32%。

鉴别器方面，我们选择原始 DCGAN 作为训练模板，训练灰度化 CIFAR-10 数据集得到鉴别器。使用灰度化的另一个原因也是为了减轻 DCGAN 鉴别器的训练压力，使其在快速训练后具有较优的鉴别能力。

(3) 基于灰度 CIFAR-10 数据集的 DCGAN 训练:

经过 200 轮的训练, DCGAN 在灰度 CIFAR-10 数据集上确实取得了一定的成效:

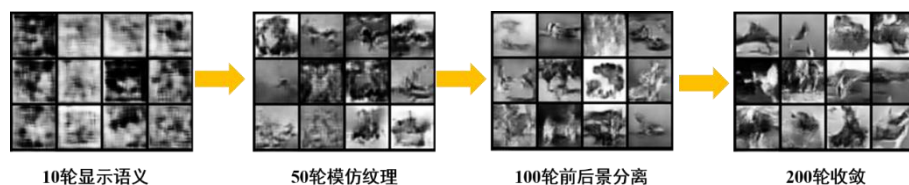


图 18 DCGAN 在灰度 CIFAR-10 数据集上的训练

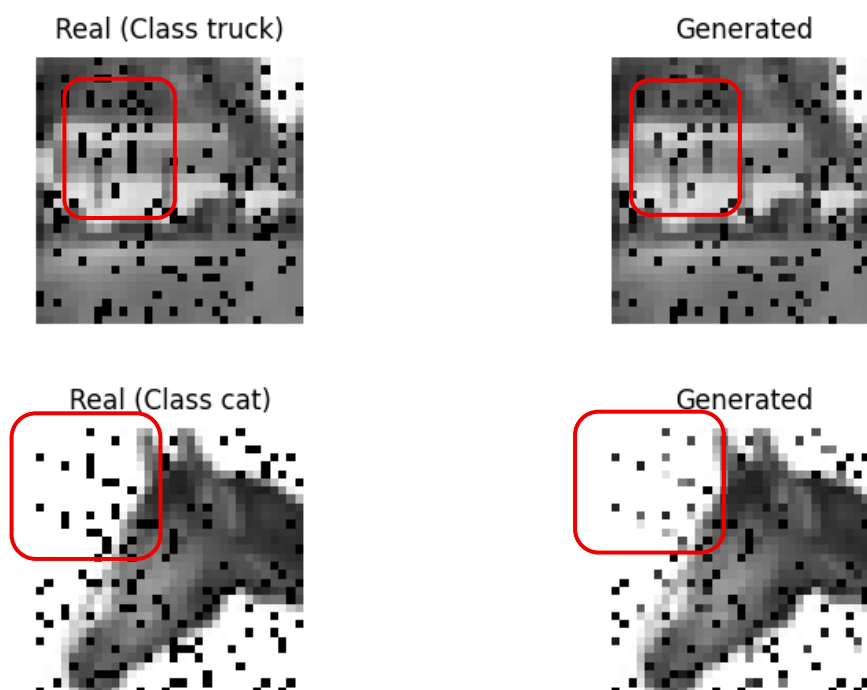
使用联合判别器可以使鉴别器参数量进一步下降, 联合判别器的参数量仅约为 DC-CGAN 的 67.35%:

表 2 条件鉴别器与联合鉴别器参数量对比

判别策略	判别器	参数量	总计
条件判别	DC-CGAN 鉴别器	1,415,777(1.42M)	1,415,777(1.42M)
联合判别	DCGAN 鉴别器	98,401(98K)	953,539(953.5K)
	ResNET56 分类器	855,138(855K)	

2. 初步训练及其结果:

确认了生成器和判别策略的设计后, 我们初步进行了 200 轮的对抗训练实验。然而, 由于时间关系, 我们仅取得了较为一般的结果。不过, 在仅有的结果中, 我们可以看出该生成器具有一定的像素填充能力:



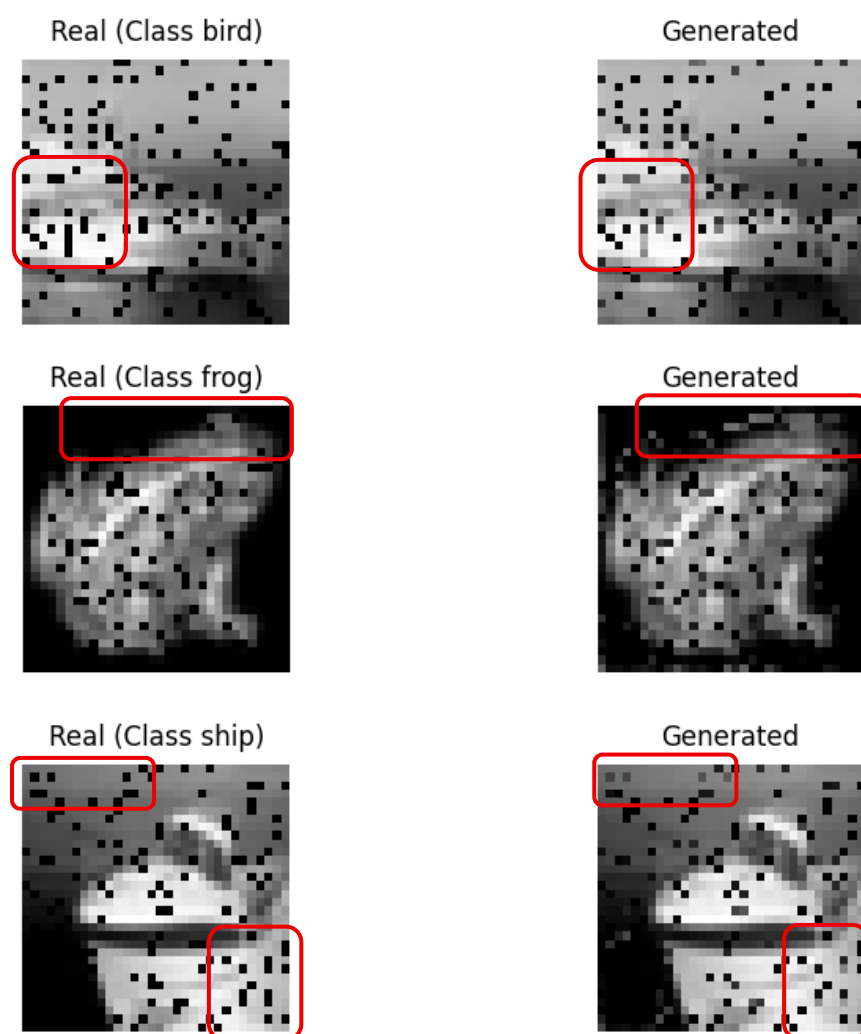


图 19 初步训练的像素填充生成器结果

3. 反思与展望：

本次训练的结果并不令人满意。因为时间的原因，我们在汇报前并没有检查问题所在。不过在汇报之后，我们深入探究了实验失败的原因，主要总结为以下两个重要部分：

(1) 联合判别能力严重不足：

我们考察了联合判别器中鉴别器和分类器的实际判别能力，发现两者完全不能满足指导生成器优化的能力：DCGAN 对灰度 CIFAR-10 数据集真实图像的判别准确率仅 59.03%；而 resnet56-cifar10 预训练网络对灰度 CIFAR-10 数据集的识别准确率仅 89.53%。这使得生成器优化过程中过多接收错误信息，实际的综合 loss 无法下降。

另一方面，联合判别器的加权损失机制也需要优化。分类损失与鉴别损失性质不同，分类错误为致命性错误，这表明生成器的工作已经影响了图像原有的特征信息，是不可接受的，惩罚力度应加强；而鉴别失误为决定性失误，这表明生成器无法胜任补充缺失像素的任务，是需要强化的，应成为主导因素。

(2) 生成器目标检测能力缺失：

目前的生成器策略虽然可以确保原有图像不受影响，但是生成器整体无法感知缺失像素位置，因此无法增强生成像素的连贯性。

同时，由于分类器的约束，生成器可能更关注分类特征，因此在留有像素的特征部位增强优化，而对缺失的背景部分缺乏优化动力。这也可能导致掩膜后的最终输出结果评价较差。

因此，后续我们将从这两点出发继续优化这一课题，我们相信经过合理的修正后可以得到一个较优的结果！

第四章 结论与展望

1. 结论

- (1) 本文围绕 CIFAR-10 数据集，设计并实现了一种基于深度卷积结构的条件对抗生成网络——DC-CGAN，旨在融合 DCGAN 的卷积结构优势与 CGAN 的条件控制能力，提升小尺寸图像的生成质量与多样性。
- (2) 生成器设计方面，本文提出三项结构创新：条件批量归一化层、加权特征融合层以及层间增强模块。这些设计不仅有效避免了模式崩溃和局部最优问题，同时显著提升了类内多样性与细节质量。与 CGAN 和 DCGAN 相比，DC-CGAN 在生成图像保真度和多样性方面均表现优异。
- (3) 训练策略方面，本文引入三批次渐进式训练流程，通过逐步调整学习率、优化早停条件和控制特征融合比例，实现了生成器与判别器在训练早期、中期与收敛阶段的稳定对抗过程，使模型更有效地收敛至纳什均衡点。
- (4) 模型评估结果表明，DC-CGAN 在 Frechet Inception Distance(FID) 与 Inception Score(IS) 两个主流指标上均优于传统模型。特别是在未显著增加参数量的情况下，相较标准 CGAN，其 FID 下降近 75%，IS 提升超过两倍，验证了本文优化策略的有效性。
- (5) 应用探索方面，本文基于 Carlini & Wagner 松弛方法设计了一种像素掩膜剥离机制，进一步延伸了条件生成网络在图像修复与前景提取方向的潜在应用，为后续构建更具语义理解能力的图像生成模型奠定基础。

2. 展望

尽管 DC-CGAN 在图像生成质量和模型训练稳定性方面已取得一定成效，但仍存在若干值得进一步研究的问题和方向：

- (1) **融合方式的可学习化：**当前的加权特征融合层采用手动设定的融合系数，后续可考虑将其设计为可学习参数，使网络能够在训练过程中自动寻优条件与噪声的融合比例，从而进一步提高图像多样性与真实性。
- (2) **判别器结构的深化与对抗机制优化：**判别器目前采用浅层卷积结构，未来可以探索多尺度或残差结构增强其判别能力，同时引入梯度惩罚、对比学习等机制，缓解判别器过拟合或失效问题。
- (3) **损失函数的扩展与组合优化：**当前损失函数仅采用二元交叉熵，后续可引入感知损失、类别一致性损失等多任务目标，增强生成图像在语义与纹理层面的综合表现力。

(4) 应用层的拓展与实际部署：除图像生成与修复之外，DC-CGAN 可进一步应用于语义图像合成、风格迁移、跨模态生成等任务，同时需探索在低资源环境中的部署方案，如模型压缩与加速推理策略。

(5) 鲁棒性与安全性研究：面对对抗样本与噪声扰动的挑战，未来还应加强对模型泛化能力与鲁棒性的研究，保障其在实际场景下的稳定性与可信度。

总而言之，本文所提出的 DC-CGAN 在轻量化设计、生成质量、多样性控制及训练策略等方面进行了多项有效探索，具备良好的应用潜力和扩展空间，期待后续工作中在更大规模数据、更复杂场景下进一步验证和完善该模型的性能。